

Year Up United

Data Analyst Training Academy:

Python Tools for Data Analysis

Week 7 Lab Workbook

This courseware is to be distributed to accompany instructor led training.

Table of Contents

MODULE 1 NUMPY	3
EXERCISE 1.A NUMPY: MANIPULATING ARRAYS.....	4
EXERCISE 1.B NUMPY RANDOM	7
EXERCISE 1.C PERFORMING OPERATIONS ON ARRAYS	9
 MODULE 2 PANDAS	 13
EXERCISE 2.A FUNDAMENTALS OF PANDAS	14
EXERCISE 2.B CLEANING UP DATA.....	17
EXERCISE 2.C WORKING WITH MULTIPLE DATAFRAMES.....	19
 MODULE 3 MATPLOTLIB	 21
EXERCISE 3.A GETTING FAMILIAR WITH MATPLOTLIB	22
EXERCISE 3.B MORE PLOTTING PRACTICE	23
 MODULE 4 SCIKIT-LEARN	 25
EXERCISE 4.A INTRODUCTION TO LINEAR REGRESSION.....	26

Module 1

NumPy

Exercise 1.A NumPy: Manipulating Arrays

Launch Jupyter Notebook from the week-07 folder, and create a new notebook named **Ex1A_NumPy_Arrays.ipynb**. Fill in the notebook with markdown and code blocks to address the questions below, and be sure to **run each cell as you go**.

1. Add a markdown cell with the name of the exercise as a header:
Exercise 1.A NumPy: Manipulating Arrays
2. Start by importing numpy (as np) into the notebook.
3. Add a markdown cell with the header ## Looping Through Arrays, then add a new code/markdown cell **for each** of the following:
 - * In a new code cell, create a variable named **arrA** as a two-dimensional array containing (11, 12, 13) and (14, 15, 16) then print `arrA`.
 - * In a new code cell, loop through the first dimension of `arrA` (i.e. *for x in arrA*) and print each element multiplied by 2.
 - * In a new code cell, loop through the second dimension of `arrA` (i.e. *for y in x*) and print each element multiplied by 2.
 - * Add a markdown cell. In your own words describe the difference between the results for first loop and the second loop.
 - * Add another code cell. Again loop through the second dimension of `arrA` (i.e. *for y in x*), but this time, assign `x*2` to a variable named `loop_result`. Add another line of code to print `loop_result`, then add a comment to the cell to explain how/why this result is different.
4. Add a markdown cell with the header ## Joining and Stacking Arrays, and add a new code cell **for each** of the following:
 - * Create variable `joined_array_1` and assign it the value of `arrA` concatenated with `arrA` on `axis=0` (the default axis). Print `joined_array_1`.
 - * Create variable `joined_array_2` and assign it the value of `arrA` concatenated with `arrA` on `axis=1`. Print `joined_array_2`.

- * Create variable `stacked_array_1` and assign it the value of `joined_array_1` stacked with `joined_array_1` on the default axis. Print `stacked_array_1`.
 - * Create variable `stacked_array_2` and assign it the value of `joined_array_1` stacked with `joined_array_2` on the default axis. What error message do you get? Add a comment to the cell to explain the error that comes back.
 - * Create variable `another_joined_array` and assign it the value of `joined_array_1` concatenated with `arrA`. Print `another_joined_array`.
 - * Create variable `another_2` and again assign it the value of `joined_array_1` concatenated with `arrA`, but this time on `axis=1`. What happens when you try to run this? Add a comment to the cell to paraphrase the error message in your own words.
 - * Create a cell with three new variables (named as you choose). In each variable, use either the `hstack()`, `vstack()`, or `dstack()` function to stack `arrA` with `arrA`. Add a comment after each that explains what the function is doing. Print the results of each variable.
5. Add a markdown cell with the header `## Splitting Arrays`, and add a new code cell **for each** of the following:
- * Create variable `splitsky` and assign it the value of `joined_array_1` split into 2. Print the variable. Then add a comment to the cell to describe the result: how many arrays? Of how many dimensions each?
 - * Create variable `spliterino` and assign it the value of `joined_array_1` split into 3. Print the variable. Then add a comment to the cell to describe the result: how many arrays? Of how many dimensions each?
6. Add a markdown cell with the header `## Searching and Replacing Array Values`.
- * In a code cell, create variable `result_1`. For the value of this variable, use `where()` to find each even number within `arrA` (hint, you'll need to write a condition using `%` and `==`). If the condition is met, print the value from `arrA`; if not met, print `'odd'`. Print `result_1`.
7. Add a markdown cell with the header `## Sorting and Filtering Arrays`, and add a new code cell **for each** of the following:

- * Create variable `arrB` as a 2-D array including the values *(7,6)*, *(13,5)*, and *(7,3)*. Create another variable, `arrB_sorted`, and assign it the sorted value of `arrB`. Print `arrB_sorted`.
 - * Finally, let's filter `arrB` using conditional logic. Start the cell with a print statement to again print the contents of `arrB`. Then create variable `z` with the value `arrB < 7` to look for values less than 7. Next create variable `arrC`, and assign it `arrB[z]`. Print `arrC`.
 - * Applying the same sort of structure, how would you create a new conditional logic filter `y` to look for values greater than or equal to 7? Follow similar steps as above to create a new variable `arrD` to hold the filtered results, and print `arrD`.
8. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes!

Exercise 1.B NumPy Random

Launch Jupyter Notebook from the week-07 folder, and create a new notebook named **Ex1B_NumPy_Random.ipynb**. Fill in the notebook with markdown and code blocks to address the questions below – and be sure to **run each cell as you go**.

1. Add a markdown cell with the name of the exercise as a header:
`# Exercise 1.B NumPy Random`
2. Start by importing numpy (as np) into the notebook.
3. Add a markdown cell with the header `## Generating Random Numbers`, and add a new code/markdown cell **for each** of the following:
 - * Create a variable named `r_1` that contains a random set of integers between 1 and 20 with a size of 2 elements. Print `r_1`.
 - * Create a variable named `r_2` that contains a random set of integers between 1 and 20 with a size of 1 row, 2 columns. Print `r_2`.
 - * Create a variable named `r_3` that contains a random set of floats between 0 and 1 with a size of 2 rows, 1 columns. Print `r_3`.
 - * Create a variable named `r_4` that contains a random set of integers between 20 and 100 with a size of 4 rows, 5 columns. Print `r_4`.
 - * Create a variable named `r_5` that contains a random set of floats between 0 and 1 with a size of 4 rows, 5 columns. Print `r_5`.
4. Add a markdown cell with the header `## Generating Random Choices from an Array`, and add a new code/markdown cell **for each** of the following:
 - * Create a variable named `students` and assign it a 1-D array containing the names of 12 of your classmates.
 - * Create a variable named `call_on` and use it to choose one random selection from `students`. Print `call_on`.
 - * Create a variable named `breakout_groups` and use it to choose a random sampling from `students` that is 3 rows, 4 columns in size. Print

`breakout_groups` and examine the results – would this be an effective way to assign breakout groups? Why or why not? Add a comment to the cell with your response.

- * Create a variable named `call_on_weighted` and use it to choose make five random selections from `students`, but this time assign different probability values that will determine how likely each student is to be selected. At least one student should have a zero likelihood of being called on, and at least one student should have a 30% likelihood of being called on. Distribute other probability values as you choose, and in a comment, explain the p values you assigned to each student. Print `call_on_weighted`.

5. Add a markdown cell with the header `## Generating a Normal Distribution`, and add a new code/markdown cell for the following:

- * Create a variable named `bell_curve` and assign it a random distribution for which the mean (peak of the curve) is 10, the standard deviation is 5, and the size of the returned array is 4 rows by 5 columns. Print `bell_curve`.

6. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes!

Exercise 1.C Performing operations on arrays

Launch Jupyter Notebook from the week-07 folder, and create a new notebook named **Ex1C_Operations_on_Arrays.ipynb**. Fill in the notebook with markdown and code blocks to address the questions below – and be sure to **run each cell as you go**.

1. Add a markdown cell with the name of the exercise as a header:
`# Exercise 1.C Performing operations on arrays`
2. Start by importing numpy (as np) into the notebook.
3. Add a markdown cell with the header `## Sample Arrays`.
 - * In a code cell, create five variables containing random sample arrays to work with, copying over the expressions you used for variables `r_1` through `r_5` in the notebook for Exercise 1.B (NumPy Random).
4. Add a markdown cell with the header `## Permutations`. Add a new code/markdown cell **for each** of the following:
 - * First, print the current value of `r_4`.
 - * Use `np.random.shuffle` to shuffle the values in `r_4`. Then print `r_4` again to view the results.
 - * Looking at the results of the previous code block, *how* were the elements of the array shuffled? Add a markdown cell to describe what you find.
 - * Create a new variable named `new_shuffle`, and assign it the value of a random permutation of `r_4`. Print `new_shuffle`, and then print `r_4` again to compare values.
 - * Add a markdown cell and briefly describe in your own words, when would you use *shuffle* versus *permutation*?
5. Add a markdown cell with the header `## Mathematical Operations`. Add a new code/markdown cell **for each** of the following:

- * Print `r_1` and `r_2` as starting values, and also print the shape of each using the `.shape` attribute. Then print the result of `r_1` plus `r_2`, using the `add` function. Also print the shape of the result.
 - * Print `r_1` and `r_3` as starting values, and also print the shape of each using the `.shape` attribute. Then print the result of `r_1` plus `r_3`, using the `add` function. Also print the shape of the result.
 - * Print `r_2` and `r_4` again as starting values, and also print the shape of each using the `.shape` attribute. Then print the result of `r_2` plus `r_4`, using the `add` function. Also print the shape of the result.
 - * Create an expression to illustrate what happens when you add `r_1` and `r_4`. What is the error message? How about adding `r_2` or `r_3` to `r_4`? Add a comment to the cell with your findings.
 - * Show what happens if you add `np.array([1])` to `r_4`.
 - * Show what happens if you add `np.array([[1]])` to `r_4`.
 - * Show what happens if you add `np.array([1,1,1,1,1])` to `r_4`.
 - * What happens if you add `np.array([[1],[1]])` to `r_4`? Add a comment to the cell with the error message this produces.
 - * Add a markdown cell, and in your own words, explain how broadcasting works to combine the values of two arrays. Using the `add` function, what gets added to what? What determines the shape of the resulting array?
6. Add a markdown cell with the header `## Rounding`. Add a new code/markdown cell **for each** of the following:
- * Print `r_3` to display the values you'll be working with for the next few cells.
 - * Round the values in `r_3` using `trunc`.
 - * Round the values in `r_3` using `fix`.
 - * Round the values in `r_3` using `floor`.
 - * Round the values in `r_3` using `ceil`.
 - * Round the values in `r_3` using `round`, to 2 decimal places.

7. Add a markdown cell with the header `## Cumulative Operations`. Add a new code/markdown cell **for each** of the following:
- * Print `r_4` to display the values you'll be working with for the next few cells.
 - * Print the sum of all values in `r_4`.
 - * Print the sum of elements along the 0 axis for `r_4`.
 - * Print the sum of elements along the 1 axis for `r_4`.
 - * Print the cumulative sum (rolling total) of values in `r_4`.
 - * Print the cumulative sum (rolling total) of values along the 0 axis for `r_4`.
8. Add a markdown cell with the header `## Grocery Problem`. Add a new code/markdown cell **for each** of the following:
- * Create two new variables, `bag1` and `bag2`. The first variable represents the contents of a grocery bag, which contains two vegetables, three fruits, four boxes of snacks, and no beverages. The second variable represents another grocery bag, which contains no fruits, four vegetables, two beverages, and one box of snacks. Assign an appropriate array to each variable that represents how many of each category of item are in each bag. Print `bag1` and `bag2`.
 - * Write an expression to calculate the total number of items in each bag, and print the result in the form of a sentence: *The first bag contains __ items, and the second bag contains __ items.*
 - * Write an expression to calculate the total number of items in each category, and print the result in the form of a sentence: *There are __ vegetables, __ fruits, __ boxes of snacks, and __ beverages in the two grocery bags.*
9. Add a markdown cell with the header `## Teamwork`. The scenario for this set of problems is that you are volunteering with a voter registration campaign, and you have been asked to oversee a squad of door-knockers. There are three people on your team: Ali, Ben, and Carla. Your squad went out to knock on doors each evening last week, Monday through Friday, and now you are assembling that data. Add a new code/markdown cell **for each** of the following:
- * Ali reported knocking on 5 doors on Monday, 4 on Tuesday, 6 on Wednesday, 4 on Thursday, and 2 on Friday. Create a variable holding an array of these values.

- * Ben missed Monday because he was out of town. He reported knocking on 5 on Tuesday, 5 on Wednesday, 3 on Thursday, and 4 on Friday. Create a variable holding an array of Ben's values.
- * Carla reported knocking on 3 doors on Monday, 6 on Tuesday, 3 on Thursday, and 4 on Friday. She was sick on Wednesday so missed that day. Create a variable holding an array of Carla's values.
- * Compile the values reported by Ali, Ben, and Carla into a single array containing all door-knocks for the week, as `all_knocks`. Print the resulting array.
- * Use `all_knocks` to calculate the total number of doors that each person knocked on for the week, and also the total number of doors that the group collectively knocked on. Print the results.
- * The organization you're working with estimates that for every two door-knocks, one of those people will actually turn out to vote. For your report to the organization, what is the rolling total of estimated votes that your group was able to turn out, day by day for last week? (Hint: Start with the rolling total by day, then multiply that array by an appropriate value to get the result.)

10. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes!

Module 2

pandas

Exercise 2.A Fundamentals of pandas

Launch Jupyter Notebook from the week-07 folder, and create a new notebook named **Ex2A_pandas_Fundamentals.ipynb**. Fill in the notebook with markdown and code blocks to address the questions below – and be sure to **run each cell as you go**.

1. Add a markdown cell with the name of the exercise as a header:
`# Exercise 2.A Fundamentals of pandas`
2. Start by importing numpy (as np) AND pandas (as pd) into the notebook.
3. Add a markdown cell with the header `## Series`. Add a new code/markdown cell **for each** of the following:
 - * Start by creating four new variables named `aa`, `ab`, `ac`, and `ad` to hold some plain Python objects.
 - Assign `aa` the list 1,2,3,4.
 - Assign `ab` the tuple 5,6,7,8.
 - Assign `ac` the set 9,10,11,12.
 - Assign `ad` a dictionary with the following key-value pairs:
'first': 13, 'second': 14, 'third': 15, 'fourth': 16
 - * Create four new variables named `ser1`, `ser2`, `ser3`, and `ser4`. Assign to each a Series created from one of the previous four variables.
 - `ser1` should use `aa`
 - `ser2` should use `ab`
 - `ser3` should use `ac`
 - `ser4` should use `ad`Print each of new Series to see the result.
 - * In a new cell, create a variable named `ser5` and assign it the literal values 17, 18, 19, 20. When you run the cell, what happens? Add a comment to the cell with the specific `TypeError` message that arises.
4. Add a markdown cell with the header `## DataFrames from Series`. Add a new code/markdown cell **for each** of the following:

- * Create new variable `df1` and give it the value of a DataFrame using `ser1`. Then on the next line, call `df1` by simply typing in the variable name (no print function). When you run the cell, a simple formatted DataFrame should be returned.
 - * Add two new columns to `df1` via the `insert()` method using `ser2` and `ser3` for each column. Call `df1` to review the updated DataFrame.
 - * Add a third new column to `df1` using `ser4`. What happens this time when you call `df1`? Add a comment explaining the difference between `ser4` compared to the other Series data used, and how this affected the outcome.
 - * Add a new row 4 using the `loc` attribute with the values 5, 9, 12, 0. Call `df1` to review the updated DataFrame.
5. Add a markdown cell with the header `## Reading Data into a DataFrame`. Add a new code/markdown cell **for each** of the following:
- * Create a new variable `df2`. Use the `read_csv` function to read the contents of `sakila_example.csv` into `df2`. (Note that you will need to save the CSV file to the same folder where Jupyter Notebook is running for this exercise.) Call `df2` to review the result.
 - * Display the top 5 records from `df2`.
 - * Display the last 10 records from `df2`.
 - * Show summary information about `df2` using the `info` method.
 - * Add a markdown cell. Looking at the `info` results, what are a few facts you can determine about this data?
 - * Show column statistics on `df2` using the `describe` method.
 - * Add a markdown cell. Looking at the `describe` results, what are the useful insights you can take away? What summarizations are likely not so useful?
 - * Create a new variable `df3`. Use the `read_csv` function to read the contents of `northwind_example.csv` into `df3`. (Note again that you will need to save the CSV file to the same folder where Jupyter Notebook is running for this exercise.) Call `df3` to review the result.
 - * Show column statistics on `df3` using the `describe` method.

- * Add a markdown cell. Looking at the *describe* results, what are the useful insights you can take away? How are the summarizations here different than for the previous example, and why?
6. Add a markdown cell with the header `## Coding a DataFrame`. Add a new code/markdown cell **for each** of the following:
- * Create a new variable named `test_data` and assign it a dictionary with the following key-value pairs:
 `'ID': [1, 2, 3, 4, 5, 6, 7, 8],`
 `'Name': ['Alicia', 'Bob', 'Charlie', 'Dominic', 'Eve', 'Frank', 'Grace', 'Heidi'],`
 `'Age': [25, np.nan, 22, 19, 31, 35, np.nan, 18],`
 `'City': ['NY', 'LA', 'Chicago', 'Houston', 'Phoenix', 'Boston', 'Austin', 'San Diego'],`
 `'Score': [85, 92, -1, 65, 78, np.nan, 55, 90]`
 - * Create a DataFrame named `df4` using `test_data`. Call `df4` to review the result.
 - * Export `df4` to a CSV file named `test_data.csv`
7. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes!

Exercise 2.B Cleaning up data

Launch Jupyter Notebook from the week-07 folder, and create a new notebook named **Ex2B_Data_Cleaning.ipynb**. Fill in the notebook with markdown and code blocks to address the questions below – and be sure to **run each cell as you go**.

1. Add a markdown cell with the name of the exercise as a header:
`# Exercise 2.B Cleaning up data`
2. Start by importing numpy (as np) AND pandas (as pd) into the notebook.
3. In a code cell, create a new DataFrame named `df` based on `test_data.csv` (as saved from the previous exercise).
 - * Add a cell to show column statistics on `df` using the *describe* method.
4. Add a markdown cell with the header `## Filtering a DataFrame`. Then add a new code/markdown cell **for each** of the following:
 - * Filter the DataFrame to show only rows where Age is greater than 18.
 - * Filter the DataFrame to show only rows where Score is greater than 50 and less than 100.
5. Add a markdown cell with the header `## Modifying a DataFrame`. Then add a new code/markdown cell **for each** of the following:
 - * Create new variable `df_cleaned`. Assign it the value of `df`, using the *drop* function to remove rows where the Score is less than 0 (i.e., invalid score data). Then call `df_cleaned` to review the result.
 - * Continuing to work with `df_cleaned` for the rest of these steps, remove the City column from the DataFrame using *drop*. Call `df_cleaned` to review the result.
 - * Replace all missing Age values with the mean Age of the DataFrame using *fillna*. Call `df_cleaned` to review the result.
 - * Replace any Score value of 0 with the mean Score of the DataFrame using *replace*. Call `df_cleaned` to review the result.

- * Remove any remaining rows that contain NaN values using the *dropna* function. Call `df_cleaned` to review the result.
 - * Apply the values from ID as index labels, then remove column ID. (Pay attention to the numbering!) Call `df_cleaned` to review the result.
6. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes!

Exercise 2.C Working with multiple DataFrames

Launch Jupyter Notebook from the week-07 folder, and create a new notebook named **Ex2C_Multiple_DataFrames.ipynb**. Fill in the notebook with markdown and code blocks to address the questions below – and be sure to **run each cell as you go**.

1. Add a markdown cell with the name of the exercise as a header:
`# Exercise 2.C Working with multiple DataFrames`
2. Start by importing numpy (as np) AND pandas (as pd) into the notebook.
3. Add a markdown cell with the header `## Basic Aggregations`. Add a new code cell **for each** of the following:

- * Create a DataFrame named `df_sales` using the following data:

```
data = {'Product': ['A', 'B', 'C', 'A', 'B', 'C', 'A', 'B', 'C'],  
        'Region': ['North', 'South', 'East', 'West', 'North', 'South', 'East', 'West', 'North'],  
        'Units_Sold': [10, 15, 20, 5, 10, 15, 10, 5, 15],  
        'Price': [100, 200, 150, 100, 200, 150, 100, 200, 150]}
```

- * Add a new column `Revenue` to `df_sales` by multiplying `Units_Sold` and `Price`.
 - * Use the `sum` function to calculate the total `Units_Sold` and total `Revenue` across all products and regions.
 - * Use the `mean` function to calculate the average `Units_Sold` and average `Revenue`.
4. Add a markdown cell with the header `## Grouping Data`. Add a new code cell for each of the following:
 - * Use the `groupby` function to group `df_sales` by `Product` and calculate the total `Units_Sold` and total `Revenue` for each product.
 - * Use the `groupby` function to group `df_sales` by `Region` and calculate the average `Units_Sold` and average `Revenue` for each region.
 - * Use the `groupby` function to group `df_sales` by both `Product` and `Region` and calculate the sum of `Units_Sold` and `Revenue` for each combination.

5. Add a markdown cell with the header `## Sorting and Ranking Data`. Add a new code cell for each of the following:
 - * Sort `df_sales` by `Units_Sold` in ascending order.
 - * Sort `df_sales` by `Revenue` in descending order.
 - * Add a new column `Revenue_Rank` to `df_sales`, which ranks the `Revenue` values in descending order (i.e., highest revenue has rank 1).
 - * Call `df_sales` to review the changes.
6. Add a markdown cell with the header `## Merging DataFrames`. Add a new code cell for each of the following:
 - * Create a new `DataFrame` named `df_products` with the following data:

```
product_data = {'Product': ['A', 'B', 'C'],
                 'Category': ['Electronics', 'Furniture', 'Appliances']}
df_products = pd.DataFrame(product_data)
```
 - * Merge `df_sales` with `df_products` on the `Product` column, resulting in a new `DataFrame` named `df_merged`.
 - * Call `df_merged` to review the merged `DataFrame`.
 - * Use *groupby* on `df_merged` to calculate the total `Revenue` by `Category`.
7. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes!

Module 3

Matplotlib

Exercise 3.A Getting familiar with Matplotlib

Launch Jupyter Notebook from the week-07 folder, and create a new notebook named **Ex3A_Plotting_Data.ipynb**. Fill in the notebook with markdown and code blocks to address the questions below – and be sure to **run each cell as you go**.

1. Add a markdown cell with the name of the exercise as a header:
`# Exercise 3.A Getting familiar with Matplotlib`
2. Start by importing numpy (as np) AND matplotlib.pyplot (as plt) into the notebook.
3. Add a markdown cell with the header `## Creating Basic Plots`. Add a new code cell for each of the following:
 - * Create two variables containing the following data points:
`x = [1, 2, 3, 4, 5]`
`y = [2, 3, 5, 7, 11]`
 - * Use `plt.plot()` to plot the line graph for `x` and `y`, and then display it using `plt.show()`.
 - * Duplicate the previous code cell, then update it to add markers to the data points using the `marker='o'` parameter.
 - * Duplicate the previous code cell, then customize the plot with a title ('Example Graph'), x-axis label ('X axis'), and y-axis label ('Y axis').
4. Add a markdown cell with the header `## Styling Plots`. Add a new code cell for each of the following:
 - * Plot the same data as above, but this time, change the line style to dashed ('--') and the line color to red.
 - * Change the marker style to a square ('s'), set the marker size to 10, and change the marker color to green.
 - * Experiment with at least two different additional marker styles and colors, and display the plots.
5. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes!

Exercise 3.B More plotting practice

Launch Jupyter Notebook from the week-07 folder, and **continue working** in the notebook **Ex3A_Plotting_Data.ipynb**. Continue adding markdown and code blocks to address the questions below – and be sure to **run each cell as you go**.

1. Add a markdown cell with the name of the exercise as a header:
`# Exercise 3.B More plotting practice`
2. Note that numpy (as np) AND matplotlib.pyplot (as plt) should already be imported at the top of the notebook. Make sure to run this cell again if you have restarted the notebook.
3. Add a markdown cell with the header `## Advanced Plotting Techniques`. Add a new code cell for each of the following:

- * Create a bar chart showing the sales of different products. Use the following data:

```
products = ['Product A', 'Product B', 'Product C', 'Product D']  
sales = [23, 17, 35, 29]
```

- * Customize your bar chart by adding a title, x-axis label, and y-axis label. Set the color of the bars to 'skyblue' and adjust the bar width to 0.5.
- * Create a scatter plot to visualize the relationship between two variables. Use the following data:

```
x = np.random.rand(50)  
y = np.random.rand(50)
```

- * Customize your scatter plot by setting the size of the markers based on another variable (e.g., `s = np.random.rand(50) * 100`), and using a colormap (e.g., `cmap='viridis'`).
4. Add a markdown cell with the header `## Subplots`. Add a new code cell for each of the following:

- * Create a figure with two subplots: one for a line chart and one for a histogram.
- * In the first subplot, plot a sine wave using:

```
x = np.linspace(0, 10, 100)
y = np.sin(x)
```

- * In the second subplot, create a histogram of 1000 random numbers generated using `np.random.randn(1000)`.
 - * Add titles and labels to each subplot.
5. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes!

BONUS: If you have time, go back through your notebooks for previous exercises using NumPy and pandas. Find examples of data that could make an interesting chart, import `matplotlib.pyplot` into the notebook, and try creating your own custom charts.

Module 4

scikit-learn

Exercise 4.A Introduction to Linear Regression

Launch Jupyter Notebook from the week-07 folder, and create a new notebook named `Ex4A_Intro_to_Linear_Regression.ipynb`.

Lab 1

1. Add a markdown cell with the name of the exercise as a header:
`# Exercise 4.A Introduction to Linear Regression`
2. Start by importing numpy (as np), pandas (as pd), matplotlib.pyplot (as plt), LinearRegression from sklearn.linear_model, and train_test_split from sklearn.model_selection.

3. Add a markdown cell with the headers:

```
## Lab 1
### Running a sample script
```

4. Open the file `linear_regression_example.py` from your course material folder. Read through the script, then copy its contents into a code cell and run it. (Do not re-import libraries that you already imported at the top of the notebook.)

You should see the output “Model trained successfully.” This confirms that the model has been fit to the training data.

5. Add a markdown cell with the header
`### Working through the sample script`
6. In a code cell, print the slope (`model.coef_[0]`) and intercept (`model.intercept_`) of the fitted model, using the following code:

```
print(f'\nSlope (coefficient): {model.coef_[0]:.2f}')
```

```
print(f'Intercept: {model.intercept_:.2f}')
```

7. In another code cell, add the following prediction code:

```
y_pred = model.predict(X_test)
```

```
print('Actual scores: ', list(y_test))
print('Predicted scores:', list(y_pred.round(1)))
```

Run the cell, and review the actual vs. predicted scores that are printed.

8. In another code cell, add the following evaluation code to find how much of the variation in your target variable is explained by your model:

```
r_squared = model.score(X_test, y_test)
print(f'R2 score: {r_squared:.3f}')
```

Run the code cell. Then add a markdown cell and answer: what does the R^2 score mean, and what does it tell you about how well this model fits the data?

9. In another code cell, add the following code to visualize the regression line:

```
# Scatter plot: actual test data points
plt.scatter(X_test, y_test, color='steelblue', label='Actual',
            zorder=3)

# Line: model predictions across the full range of X
x_range = np.linspace(
    X['hours_studied'].min(),
    X['hours_studied'].max(),
    100).reshape(-1, 1)
y_line = model.predict(x_range)
plt.plot(x_range, y_line, color='tomato', label='Regression line')

plt.xlabel('Hours Studied')
plt.ylabel('Exam Score')
plt.title('Linear Regression: Hours Studied vs. Exam Score')
plt.legend()
plt.tight_layout()
plt.show()
```

Run the code. Then add a markdown cell and describe what you see: how closely do the actual data points follow the regression line?

Lab 2

10. Continue working in Ex4A_Intro_to_Linear_Regression.ipynb. Add a markdown cell with the headers:

```
## Lab 2
```

```
### Building your own model
```

11. In a code cell, create a DataFrame named `df_ads` and assign to it the following data:

```
ad_data = {
    'monthly_ad_spend': [500, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500,
                        5000, 5500, 6000, 6500, 7000, 7500, 8000, 8500, 9000, 9500, 10000],
    'monthly_revenue': [4200, 5100, 6800, 7400, 8900, 9200, 10500, 11800,
                       12400, 13100, 14200, 15600, 15900, 17200, 18100, 19400, 20200, 21500, 22100,
                       23800]
}
```

Call `df_ads` to review the data.

12. Define your feature matrix `X2` using the `monthly_ad_spend` column (use double brackets to keep it as a DataFrame), and your target vector `y2` using the `monthly_revenue` column. Print the shape of `X2` and the shape of `y2`.
13. Split `X2` and `y2` into training and test sets using `train_test_split`. Use `test_size=0.2` and `random_state=42` to match the approach from Lab 1.
14. In a separate cell, create a LinearRegression model named `model2`, fit it to the training data, and print “Model trained successfully.”
15. Print the slope (`model2.coef_[0]`) and intercept (`model2.intercept_`) of the fitted model. Add a comment interpreting what the slope means in the context of ad spend and revenue (e.g. “for each additional __, the predicted __ increases by __”).
16. Add a markdown cell with the header `### Making predictions and evaluating the model`.
17. Use the fitted model to predict revenue for the test set. Print the actual values and the predicted values side by side (rounded to 2 decimal places).
18. Calculate and print the R^2 score for the test set. Then add a markdown cell and give your interpretation of the result: does the model appear to be a good fit? What percentage of the variation in revenue is explained by ad spend?
19. Add a markdown cell with the header `### Visualizing the results`.

20. Create a scatter plot of the full dataset (all x_2 values vs. all y_2 values). Label the x-axis 'Monthly Ad Spend (\$)', the y-axis 'Monthly Revenue (\$)', and give the plot the title 'Ad Spend vs. Revenue'. *Don't include a regression line yet!* Run just the scatter plot (dots only).
21. In another cell, copy and paste the code you used to generate a scatter plot. This time, add a regression line overlayed on the scatter plot. Use a contrasting color and add a legend distinguishing the data points from the regression line. Then run the cell to display the plot.
22. Add a markdown cell and reflect: Does the regression line appear to fit the data well visually? Does this match what the R^2 score told you?
23. BONUS: Go back to step 12 and change the settings for `test_size` and `random_state` to experiment with different values (e.g. `test_size=.3` and `random_state=10`). Re-run the following code cells to see how that effects the results. Add another markdown cell at the bottom of the notebook to share your observations.
24. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes!

End of Week 7 Lab Workbook